

Implicit Neural Representations

Cross-Modal Steganography — Phase III Technical Reference

B.Tech Final Year Project · Group 02 · IUST Kashmir · Semester 8

SIREN Architecture · Key-Based Extraction · Multi-Modal Support · Adaptive Compute

1. What is an INR?

An Implicit Neural Representation (INR) encodes a continuous signal — an image, audio waveform, or video — as the weights of a neural network rather than as discrete samples. The network learns a function $f(x) \rightarrow \text{value}$, where x is a coordinate and value is whatever the signal carries at that position (RGB, amplitude, etc.).

The fundamental paradigm shift from traditional storage to INR:

Property	Traditional (Pixels)	INR (Network Weights)
Storage unit	Discrete pixels / samples	Floating-point network weights
Resolution	Fixed at capture time	Continuous — query any coordinate
File size	Grows with resolution	Fixed by architecture, not content
Compression	Lossy (JPEG) / lossless (PNG)	Implicit via network capacity
Steganography	Pixel-level manipulation	Weight-space entanglement

"The network weights are the image. There are no pixels — only a function."

2. Why SIREN for Steganography?

Standard ReLU networks fail to capture high-frequency content in signals because the ReLU activation and its derivatives are piecewise constant — they cannot represent smooth oscillations. SIREN (Sinusoidal Representation Networks) solves this by replacing every activation with a sine function:

SIREN activation: $\phi(x) = \sin(\omega_0 * W * x + b)$ where $\omega_0 = 30$ amplifies the frequency spectrum, enabling the network to represent fine textures, sharp edges, and subtle steganographic perturbations that standard networks miss.

For steganography, SIREN's ability to faithfully reconstruct high-frequency signals is essential — it means the hidden modality can be encoded with sufficient precision to be meaningfully recovered during extraction.

3. Cross-Modal INR Architecture

The cross-modal system extends the basic SIREN with a dual-head design. One shared backbone encodes representations for both modalities simultaneously. The cover modality is queried openly; the secret modality is locked behind a key-derived coordinate shift.

Layer	Component	Output Shape	Notes
Input	Coordinate grid (x,y) or (x,y,t)	(N, in_dim)	Normalised to [-1, 1]
Layer 0	SIREN Layer — first	(N, hidden_dim)	Uniform init [-1/in, 1/in]
Layers 1..L-2	SIREN Layers — hidden	(N, hidden_dim)	Init with $\sqrt{6/in}/\omega_0$
Branch A	Linear + Sigmoid — cover head	(N, out_dim_1)	RGB / amplitude for cover
Branch B	Linear + Sigmoid — secret head	(N, out_dim_2)	Queried with key-shifted coords
Output	Reconstructed modality signal	Reshaped to original dims	Both heads share backbone

Loss Function

Training minimises a weighted combination of reconstruction losses across both heads:

$L_{\text{total}} = 0.6 * \text{MSE}(\text{cover_pred}, \text{cover_gt}) + 0.4 * \text{MSE}(\text{secret_pred}, \text{secret_gt})$ The 60/40 weighting slightly favours cover fidelity to maintain visual/perceptual imperceptibility of the stego carrier.

Key System

The integer key seeds a NumPy RNG which generates a deterministic coordinate shift vector. Secret coordinates are perturbed by this vector before feeding into the network, then clamped via $\tanh()$:

key_system.py

```
# Key to coordinate shift
def key_to_shift(key: int, coord_dim: int):
    rng = np.random.RandomState(key)
    shift = rng.uniform(-0.5, 0.5, coord_dim)
    return torch.from_numpy(shift.astype(np.float32))

# Apply shift (tanh keeps coords in [-1, 1])
def apply_key(coords, shift):
    return torch.tanh(coords + shift.unsqueeze(0))
```

Without the correct key, the queried coordinates map to an entirely different region of the learned function — extraction returns perceptually random noise rather than the hidden signal.

4. Supported File Formats & Specifications

4.1 Image Modality

.png

.jpg

.jpeg

.bmp

.webp

.tiff

Parameter	Default	Override	Notes
Max resolution	128 x 128 px	--max_size N	Resized proportionally (LANCZOS)
Colour space	RGB (3 channels)	—	Alpha channel stripped on load
Coordinate space	[-1, 1] x [-1, 1]	—	Normalised 2D grid (x, y)
INR input dim	2	—	(x, y) coordinate pair
INR output dim	3	—	R, G, B in [0, 1]
Library	Pillow	—	pip install Pillow

4.2 Audio Modality

.wav

.mp3

.flac

.ogg

.aac

Parameter	Default	Override	Notes
Max samples	22,050	Edit load_audio_modality.py	Default @ 22,050 Hz sample rate
Channels	Mono (1)	—	Stereo auto-mixed to mono by librosa
Normalisation	[0, 1]	—	Min-max normalised; stats saved for denorm
Coordinate space	[-1, 1] (1D)	—	Linear time axis t
INR input dim	1	—	Time coordinate t
INR output dim	1	—	Amplitude in [0, 1]
Libraries	librosa + soundfile	—	pip install librosa soundfile

4.3 Video Modality

.mp4

.avi

.mov

.mkv

.webm

Parameter	Default	Override	Notes
Max frames	30	Edit load_video_modality.py	Frames beyond limit truncated
Frame size	64 x 64 px	--max_size N	Resized via cv2.resize
Total points	30 x 64 x 64 = 122,880	—	Heavy on CPU — use quality=low
Coordinate space	[-1,1]^3	—	3D grid (x, y, t) — spatial + temporal

Parameter	Default	Override	Notes
INR input dim	3	—	(x, y, t) triple
INR output dim	3	—	R, G, B per frame pixel
Library	opencv-python	—	pip install opencv-python

4.4 Text Modality

.txt

Parameter	Value	Notes
Max characters	1,024	Longer text accumulates float-rounding errors
Encoding	Normalised char index $\text{ord}(c) / 127.0$	Maps printable ASCII to $[0, 1]$
Coordinate space	$[-1, 1]$ (1D)	Character position axis
INR input dim	1	Character index t
INR output dim	1	Normalised char value
Charset	UTF-8 (printable ASCII best)	Non-ASCII chars clamp to range $[32, 126]$
Library	Built-in (no extra pip)	—

5. Valid Modality Combinations

Any two different modalities can be paired. The table below shows valid combinations, coordinate dimensionality, and relative GPU/CPU memory cost:

Cover (Modal 1)	Secret (Modal 2)	In Dims	Points (default)	Cost	Supported
Image	Audio	2 / 1	16,384 + 22,050	Low	Yes
Image	Text	2 / 1	16,384 + 1,024	Low	Yes
Image	Image	2 / 2	16,384 + 16,384	Low	Yes
Audio	Text	1 / 1	22,050 + 1,024	Low	Yes
Audio	Audio	1 / 1	22,050 + 22,050	Medium	Yes
Video	Image	3 / 2	122,880 + 16,384	High	Yes
Video	Audio	3 / 1	122,880 + 22,050	High	Yes
Video	Video	3 / 3	245,760+	Too heavy	No

6. Quality Presets

Quality presets scale the network architecture and training duration to match available hardware. The system auto-selects a preset on startup if `--quality` is not specified:

Preset	Steps	Hidden Dim	Layers	Learn Rate	Target Hardware	Est. Time (CPU)
low	1,000	128	4	2e-4	CPU, any machine	2-4 min
medium	2,000	256	5	1e-4	Modern CPU / weak GPU	8-15 min
high	5,000	512	6	5e-5	GPU (GTX 1060 / 4GB+)	3-6 min (GPU)
ultra	10,000	1,024	7	2e-5	Research GPU (8GB+)	10-20 min (GPU)

7. CLI Reference

Flag	Type	Default	Description
<code>--modal1</code>	str	None	Cover modality file path
<code>--modal2</code>	str	None	Secret modality file path
<code>--mode</code>	str	hide	hide or extract
<code>--key</code>	int	42	Integer seed for coordinate shift
<code>--quality</code>	str	auto	low / medium / high / ultra
<code>--weights</code>	str	None	siren_weights.pth path (extract only)
<code>--output</code>	str	output/	Directory for all saved outputs
<code>--max_size</code>	int	128	Max spatial resolution (image/video)

Usage examples:

cross_modal_inr.py — example commands

```
# Hide audio inside image (auto quality)
python cross_modal_inr.py --modal1 cover.png --modal2 secret.wav --mode hide --key 42

# Hide text inside image, high quality
python cross_modal_inr.py --modal1 cover.png --modal2 msg.txt --mode hide --key 99 --quality high

# Extract secret back with correct key
python cross_modal_inr.py --modal1 cover.png --modal2 secret.wav \
    --mode extract --key 42 --weights output/siren_weights.pth

# Video cover, image secret, low quality (CPU-safe)
python cross_modal_inr.py --modal1 clip.mp4 --modal2 hidden.png --mode hide --key 7 --quality low
```

8. Output Files

File	Description	Notes
<code>cover_reconstructed.*</code>	Cover modality rebuilt from INR weights	Extension matches input file
<code>secret_reconstructed.*</code>	Secret modality rebuilt (correct key only)	Wrong key → noise output
<code>siren_weights.pth</code>	The steganographic carrier — weights encode both modalities	Shouldn't be used to transmit the secret
<code>report.json</code>	Metrics: PSNR, params, timing, key hash	Key hash is SHA-256 — key not stored

9. Dependencies

Package	Version	Required For	Install
torch	>=1.12	Core INR training	See PyTorch.org for CUDA variant
numpy	>=1.23	Array ops, key generation	pip install numpy
Pillow	>=9.0	Image load/save	pip install Pillow
librosa	>=0.10	Audio load (MP3, FLAC, OGG)	pip install librosa
soundfile	>=0.12	Audio save (WAV, FLAC)	pip install soundfile
opencv-python	>=4.7	Video load/save	pip install opencv-python
scikit-image	>=0.20	SSIM metric	pip install scikit-image
tqdm	>=4.65	Progress bars (optional)	pip install tqdm

10. Evaluation Metrics

The following metrics are computed automatically after training and saved to report.json:

Metric	Formula	Good Threshold	Meaning
PSNR (Cover)	$10 * \log_{10}(1 / \text{MSE})$	> 30 dB	Perceptual quality of reconstructed cover
PSNR (Secret)	$10 * \log_{10}(1 / \text{MSE})$	> 25 dB	Fidelity of recovered secret signal
SSIM (Image)	Structural Similarity Index	> 0.85	Perceptual structural quality (images only)
Params / Pixels	Total params / (H*W*C) < 1.0 = compressed		Compression efficiency of the INR
Training time	Wall-clock seconds	Logged per run	Useful for hardware benchmarking

11. Phase Roadmap

Phase	Semester	Focus	Status
Phase I	Sem 6	Classical LSB Steganography	Complete
Phase II	Sem 7	Deep Learning — CNN Steganography	Complete
Phase III	Sem 8	Cross-Modal INR Steganography	In Progress
Phase III+	Sem 8	Steganalysis resistance, GAN-based training	Upcoming

Phase	Semester	Focus	Status
Phase IV	Sem 8	Research paper, evaluation, submission	Upcoming



The weights file is the secret. The coordinates are the lock. The key is yours.

Cross-Modal INR Steganography — Group 02 — IUST Kashmir — 2025